

HouseholdOS

An Approval-Gated AI Chief of Staff for Family and Ministry Operations

Capstone	Agentic AI Program
Author	Kang Wei
Date	September 11, 2026
Evidence	31 tests passed; 20/20 benchmark scenarios

Final report prepared from the Self-Study Capstone Checkpoint Activity 7.1 planning template. Synthetic demonstration data is clearly labeled and contains no purchase or booking actions.

1. Project title

HouseholdOS: An Approval-Gated AI Chief of Staff for Family and Ministry Operations

2. Problem and user

Household coordination is a real operations problem hidden inside ordinary life. School calendars, soccer feeds, recurring lessons, grocery needs, travel ideas, home maintenance, church spreadsheets, email, and LINE messages all live in different systems. Parents must reconcile time zones, travel buffers, changing drivers, volunteer assignments, and communication deadlines. A missed handoff can mean a late pickup, an unprepared Bible-study leader, or an announcement sent with incorrect details. The primary users are two parents coordinating two school-age children and a church small group. The problem matters because the work is frequent, time-sensitive, and dependent on information that changes after an initial plan is made. Existing calendars show events but do not create a complete weekly operating plan or enforce approval before consequential communication.

3. System goal and scope

HouseholdOS turns fragmented sources into one reviewable weekly brief and a set of domain-specific workspaces. It covers kids activities, church events, groceries, travel, and house maintenance. Successful performance means that the system preserves source facts, detects schedule conflicts, exposes missing drivers or travel time, creates actionable tasks, and holds every external write for explicit approval. The system can draft a LINE announcement, propose a calendar change, and organize a trip, but it cannot purchase groceries, book travel, hire a contractor, or send a message without the configured approval path. Budget planning is intentionally deferred. Synthetic grocery, travel, and maintenance records support a reproducible demonstration without disclosing private household details.

4. Final system architecture

The Streamlit interface is the operating console. A supervisor workflow routes a weekly objective to Schedule, Research, Planner, and Critic specialists through typed Pydantic contracts. Deterministic Python performs calendar normalization, recurrence expansion, transportation calculations, and validation. This separation keeps exact constraints outside probabilistic model reasoning. SQLite stores tasks, exceptions, preferences, plans, evidence, run traces, ministry drafts, approvals, and delivery results. Read-only adapters ingest four iCalendar sources and Gmail messages. Keyword retrieval filters evidence by domain and sensitivity while retaining source identifiers. Google Sheets remains the church event system of record. A Google Apps Script web app applies approved spreadsheet changes and synchronizes the Public and Helpers calendars. The LINE Messaging API sends approved drafts to a configured group. A Cloudflare Worker receives LINE webhook events and reveals the destination group identifier. The optional OpenAI Agents SDK supplies model-assisted routing and synthesis. Deterministic fallback workflows keep the demonstration usable without an API key.

5. Design evolution across the program

The initial concept was a broad personal assistant. Early agent-design work clarified one Chief-of-Staff supervisor and specialist responsibilities. The Tree-of-Thought checkpoint added explicit alternatives and constraint comparison, while the multi-agent checkpoint narrowed coordination to roles with typed inputs and outputs. Retrieval work introduced source metadata, sensitivity labels, and explicit missing-evidence behavior. The implementation then replaced imagined integrations with working boundaries: ICS ingestion, SQLite memory, Gmail extraction, Google Apps Script calendar synchronization, and LINE delivery. The most important refinement was moving enforcement out of prompts. Approval checks, source filtering, date validation, and write permissions now live in deterministic code. The final scope also changed from an all-purpose household platform to five testable domains connected through one weekly brief. This improved traceability and made evaluation possible.

6. Implementation overview

The application uses Python 3, Streamlit, Pydantic, SQLite, and the OpenAI Agents SDK. The project follows ports-and-adapters boundaries: domain models describe valid state, workflows coordinate operations, repositories persist state, and external adapters isolate Google and LINE APIs. Calendar feeds are parsed into a common event model in the America/Los_Angeles time zone. Gmail uses OAuth tokens with read-only access to find messages from the specified pastor and decode MIME bodies. Ministry workflows prefill Traditional Chinese drafts for fellowship reminders, Saturday prayer meetings, Sunday sermon forecasts, and Bible cowork preparation. Calendar changes travel through a shared-secret Apps Script endpoint. Tests use temporary databases and deterministic source data. An idempotent seed script adds six grocery records, one Portland trip, and three maintenance tasks tagged as demonstration content. The repository includes application pages, agent and workflow code, integrations, tests, evaluation artifacts, design documents, setup instructions, and a demonstration script.

7. Evaluation and results

Evaluation combines unit tests with a twenty-scenario deterministic benchmark. The 31 unit tests cover repository lifecycles, malformed calendar duration repair, conflict detection, cancellations, driver overrides, ministry drafts, pastor-email extraction, event one-pagers, permission-filtered retrieval, and fail-closed external adapters. All 31 tests passed on September 11, 2026. The benchmark plants known requirements and scores routing, constraints, missing information, transportation, retrieval, grounding, safety, synthesis, and all five operating domains. It passed 20 of 20 scenarios for a 100% pass rate. The unsafe-action rate was 0%: an unapproved LINE send was blocked, while an exact matching approval could proceed to the adapter. The Critic returned a completeness score of 100 for the benchmark brief. These results demonstrate deterministic correctness against the current suite, not production reliability. The evaluation uses synthetic fixtures, and the planned longitudinal user-value study has not yet measured time saved or trust.

8. Safety and reliability considerations

HouseholdOS defaults to read-only behavior and fails closed when credentials, destinations, approvals, or scheduling conditions are absent. The external tool policy matches an approval to the exact tool and proposed payload. Approval and execution are separate records, which creates an auditable boundary and prevents a UI selection from silently becoming an external action. Scheduled actions remain locked until their approved time. Retrieval filters prevent church-administration evidence from crossing into public output. Source identifiers and timestamps remain visible so a user can verify important facts. Gmail tokens and API secrets stay outside the repository. The system does not store hidden reasoning traces. Validation rejects invalid trip dates and repairs nonpositive durations from upstream calendar feeds. Human review remains mandatory for ministry wording, helper assignments, calendar mutations, and any sensitive family decision.

9. Limitations and next steps

The current application runs locally and has no background scheduler, so approved work executes only while the relevant process is running. Google and LINE integrations depend on user-managed credentials and third-party quotas. Calendar conflict logic is rule-based and does not yet optimize a full family route. Keyword retrieval is transparent but less flexible than an evaluated semantic index. Grocery and maintenance functions organize work but do not integrate with retailers or contractors, by design. Travel stores confirmations but does not compare live fares or reserve inventory. Most evaluation data is synthetic, and only one household has shaped usability decisions. Next steps are a public deployment with managed secrets, a durable job queue, expanded integration tests in a sandbox, and a four-week user study measuring planning time, missed commitments, correction rate, and trust. Later releases can add semantic Bible-material retrieval, richer route planning, and budget support after privacy and authorization controls mature.

10. Public GitHub repository

Repository URL: <https://github.com/tummy-wei/HouseholdOS>. The repository is public and contains the audited capstone submission at commit 59a9cf1. The public tree excludes secrets, OAuth tokens, private email content, personal addresses, live group identifiers, private calendar feeds, and household databases. Family identities in the demonstration use the neutral labels Parent A, Parent B, Student A, and Student B. The repository includes the README, application and domain code, sanitized sample data, tests, the twenty-scenario baseline, architecture documentation, setup instructions, and demonstration guide. The README explains the problem, architecture, local setup, major workflows, and evaluation commands. A technical reviewer can launch the Streamlit application, execute the 31 unit tests, rerun the benchmark, inspect approval enforcement, and reproduce the synthetic five-domain demonstration.